

Coastal water that behaves like a beach

DnDRom engineering research | 7 September 2026

Audience: DnDRom's product owner and rendering implementation. Scope: locally generated UE 5.8 coastlines, beach surf, foam, caustics and their streaming/performance constraints. Hardware target: 8 GB VRAM and 16 GB system RAM. This is a research and implementation specification, not a release announcement.

Decision

Replace the current shoreline-band effect with a coordinated coastal system: bathymetry-driven wave motion, a surface that can run onto the beach, persistent transported foam, and lighting evaluated on underwater receivers. Keep Single Layer Water for the water surface and volume response. Build and measure the coastal system in a bounded neighborhood; do not simulate an infinite ocean at beach resolution.

This recommendation is an engineering synthesis of production approaches, not a claim that one paper or plugin supplies a ready-made DnDRom solution. The current water candidate remains visually rejected. Its successful compilation does not establish beach realism.

Why the current effect fails

Foam is computed afresh from shore distance and time. A periodic crest function produces synchronized white contours around the coastline; noise breaks their edges but does not give them a physical history. There is no stored foam density, birth event, transport or decay. Consequently the pattern follows the map contour instead of visibly remaining behind a breaking wave.

The local water exporter clips triangles at signed depth zero. Surface displacement fades toward that boundary. Water therefore cannot wash above the fixed mean shoreline and retreat from it. The distant exporter also terminates its mesh at a fixed wetness contour. A more detailed foam texture cannot repair either geometric limitation.

Caustics are currently evaluated at the water surface's world XY coordinates. A moving cell-edge network and wave-curvature estimate modulate transmitted color there. This is an artistic pattern, not refracted illumination attached to the submerged terrain. A stationary image containing thin lines would not, by itself, fix that coordinate-space error.

Local evidence inspected: `scripts/unreal/water_surface.hls`; `scripts/unreal/prepare_content.py`, `make_water`; `apps/desktop/src/migration/unrealScene.ts`, `waterMesh`; `apps/desktop/src/migration/nativeAtlasWater.ts`. The latest automatic beach-close capture is dominated by foliage, so it cannot serve as a meaningful close-up water acceptance image.

What the production evidence actually demonstrates

Horizon Forbidden West separates the animated breaking-wave cross-section from the paths and timing that fit it to a coast. Its final cross-section was hand animated after an initial simulation needed substantial cleanup. Shape, guide and animation curves control propagation and breaking; foam appears after breaking. Those authored paths were baked and streamed with world tiles. This establishes a useful hybrid design, but does not demonstrate automatic fitting to arbitrary runtime terrain. [Hugh Malan, Guerrilla, Rendering Water in Horizon Forbidden West, SIGGRAPH 2022, slides 9–20.](#)

Arden's Wake tracks whitewater as a scalar field: fluid acceleration creates it, velocity transports it, and dissipation removes it. Its foam field could use higher resolution than the underlying water simulation. The talk also reports precision and resolution limits for its shallow-water solver; its successful configuration is not a transferable hardware budget for DnDRom. [Devon Penney, Penrose, Water in Arden's Wake, GDC 2018, slides 35 and 43–44.](#)

Crest's implementation independently confirms the same persistent-state principle. It samples previous foam at a flow-offset world position, handles changing simulation cascades, fades existing density and injects new foam from displacement compression and shallow water. Its source header identifies MIT licensing; this is a Unity implementation, not a UE integration. The design recommendation here does not require copying its code. [Wave Harmonic, UpdateFoam.compute, live master source accessed 7 September 2026.](#)

From Shore to Horizon addresses beach contact through corresponding water/beach topology, shallow bathymetry and coordinated displacement, with wet-sand and foam treatment. It is a practical coastal rendering reference, not proof that a depth-faded ocean plane produces convincing crashing surf. [Nicolas Longchamps, Eidos-Montreal LABS, From Shore to Horizon, GDC 2017, slides 54–65.](#)

Portal 2's flow-map technique alternates short texture-advection phases and blends between them, with phase variation to reduce pulsing. It is useful for small moving foam detail. It transports a visual pattern; it must not be confused with preserving the amount of foam across frames. [Alex Vlachos, Valve, Water Flow in Portal 2, SIGGRAPH 2010.](#)

Choice for DnDRom

Use a hybrid coastal renderer first, with explicit limits. A bounded shallow-water solver is an alternative when interaction and refraction of wave fronts justify its integration cost. A heightfield alone cannot represent an overturning lip; research adds separate breaking-wave geometry for that reason. Large curling breakers would require that additional representation and should not be claimed from a displaced plane. [Thuerey and colleagues, Real-time Breaking Waves for Shallow Water Simulations, 2007.](#)

Concrete runtime design

The following is the proposed DnDRom implementation, not functionality already delivered. Keep the established terrain generation, site selection and erosion cache intact. Generate coastal inputs from the same authoritative terrain that is rendered, so the town and streamed surroundings do not acquire separate shorelines.

1. Coastal data. Supply signed bed depth, mean water elevation, shore distance, a shore normal and exposure to incoming waves. Distance must remain in metres and must not be inferred from normalized depth. Include a narrow dry-side band so run-up has geometry and terrain data to use. Resolve overlapping tile inputs using the finest displayed authoritative surface, with a halo and a deterministic transition rule.
2. Wave motion. Separate long open-water swell from fine normal detail. For the first beach implementation, fit finite spilling-wave profiles to the coast, with a coherent incoming direction, variable arrival times and independently varying breaking age along each front. Reduce energy in sheltered areas. Do not make every shoreline emit the same periodic wave. Coast fitting and corner behavior are prototype questions; the cited authored tools do not solve them automatically.
3. Moving contact. Use the same wave height and phase for vertex displacement, wave normals, instantaneous wet/dry coverage and foam generation. Extend only the narrow coastal mesh beyond mean water level, then mask or clip it against the displaced water height and authoritative bed. Refine geometry where a moving crest is visible, rather than increasing resolution over the whole lake. A wet-sand state should remain briefly after retreat and fade; it must not paint dry terrain permanently.
4. Persistent foam. Maintain two ping-pong density textures in a bounded world-space window. Each fixed update backtraces through the flow field, samples previous density, dissipates it and adds new density at breaking fronts. Transport and decay remain active when new emission stops. Carry world coordinates and previous origins through a camera-window shift; reproject retained state instead of resetting it on camera movement. Newly exposed regions need bounded initialization and a fade, not an abrupt white border.
5. Foam appearance. Resolve broad coverage from the persistent density. Add smaller advected detail with holes, connected strands and irregular patches. Use higher roughness and a stronger surface contribution where foam actually occupies the pixel. Treat submerged bubbles separately from surface foam. Let large coverage survive minification while filtering subpixel bubble detail. A visually dense white line is not the acceptance target.
6. Streaming and ownership. One world-level controller owns simulation time, textures and water parameters. Tile actors register validated coastal fields and release them when destroyed; they do not each restart a water simulation. Scene changes clear the correct world's state. Tile replacements and LOD transitions preserve it. Native imports must bound field dimensions, finite values and memory, just as mesh imports already do.

Integration points

The exporters need signed coastal data and a dry-side mesh strip. A native water controller should own the persistent textures and a shared dynamic material. `prepare_content.py` should cook separate update and surface materials; `water_surface.hlsl` should sample state rather than synthesize shoreline bands. The existing material compilation guard remains necessary. Add diagnostics for simulation dimensions, update cost, active field count, state reprojections and invalid-input rejection before measuring visual quality.

Caustics and underwater contact

Caustics are focused illumination received by submerged surfaces. Their appearance depends on the refracting surface, illumination and receiver geometry. NVIDIA's GPU Gems chapter presents an efficient approximate construction using wave behavior and receiver information, and explicitly distinguishes such approximations from more complete light simulation. That is a better foundation than naming an animated cellular texture "physical caustics." [Juan Guardado and Daniel Sanchez-Crespo, Rendering Water Caustics, GPU Gems, NVIDIA, 2004.](#)

Single Layer Water already provides scattering, absorption, reflection and refraction using depth and color behind the water surface. Its Color Scale Behind Water input changes the transmitted contribution; it does not automatically reconstruct a receiver or generate a caustic field. Retain this shading model, but supply the appropriate data. [Epic Games, Single Layer Water Shading Model, UE 5.8 documentation, accessed 7 September 2026.](#)

The installed UE 5.8 shader additionally fades ColorScaleBehindWater using water-volume depth multiplied by 0.02, reaching full strength at 50 cm along the relevant depth measure. That helps explain why merely increasing the current multiplier is an unreliable way to add very shallow shoreline detail. This is a verified implementation detail of the installed engine, not a portable promise about every UE version.

For an initial inexpensive approximation, reconstruct the visible receiver using SceneDepthWithoutWater and the camera projection, evaluate the moving wave-derived focusing pattern there, and attenuate it with actual receiver depth and lighting. Reject sky/fallback depths and unsupported views. Use separate orthographic reconstruction if that camera mode is added. Sampling an unrefracted receiver will not exactly follow Single Layer Water's internally distorted UVs; that mismatch must be tested and disclosed.

For stronger optical consistency, prototype a bounded caustic light map: refract sunlight through samples of the same animated wave surface, intersect the authoritative bathymetry, accumulate filtered contributions, and sample the result on underwater receivers. This is a proposed non-RTX engineering path, not an implementation supplied by the current shader. Receiver occlusion, steep beds, filtering and temporal stability remain work. Do not make ray-traced caustics a minimum-hardware requirement.

Installed-source checks: HLSLMaterialTranslator.cpp, SceneDepthWithoutWater; MaterialTemplate.usf, MaterialExpressionSceneDepthWithoutWater; SingleLayerWaterShading.usf, ColorScaleBehindWater. The depth expression is supported in the relevant pixel shading paths, not the vertex shader. It returns converted view depth, not world-space Z.

What must remain visually connected

Surface displacement, water normals, foam emission and caustics must use one clock and compatible wave parameters. A submerged rock should receive moving light that stays attached while the camera orbits. Foam belongs on the moving surface; wet sand belongs on exposed terrain. Wind and sun changes should alter the appropriate effects, rather than simply scrolling every texture faster.

Budget and quality gates

Proposed starting allocation: two 512 by 512 R16F foam textures require 1 MiB of raw texel storage. One RGBA16F coastal-input texture of that size requires 2 MiB. A 256 m-wide window gives 0.5 m density cells, so it cannot directly resolve individual bubbles; finer appearance belongs in filtered shading. These are calculated storage figures, excluding resources, transient copies, caustic maps and other renderer overhead. They are not measured GPU cost or proof of the 8 GB hardware target.

Start foam updates at 30 Hz with interpolation in rendering. Use bounded substeps and an explicit policy for stalls; never accumulate an unlimited catch-up queue. Test a 256-square fallback before adopting it. Measure GPU timings, CPU field updates, uploads and peak memory separately. Compare identical cameras and settings on an otherwise idle machine; current shared-load screenshots do not establish performance on an 8 GB / 16 GB PC.

Acceptance requires motion. Capture at least 20 seconds at a clear low beach view and a matching overhead view. Observe approach, steepening, breaking, run-up and retreat, with patchy residue that continues moving and dissolves. Disable new emission during a test: existing foam must remain temporarily, advect and disappear gradually. It must not vanish immediately or persist forever.

Orbit a stationary submerged object to verify receiver attachment; compare sunlit and shaded shallows, deeper water and the dry beach. Avoid regular dots, persistent grid lines, bright caustics on dry ground and light patterns that track the camera. Inspect temporal shimmer as well as a still frame. A screenshot hidden by trees fails the test setup, regardless of its automated pass flags.

Stream across tile boundaries, move the camera-window origin, pause and resume, unload a tile and switch campaigns. Check that foam state does not reset during ordinary navigation, coastlines remain continuous and memory stays bounded. Exercise a sandy beach, sheltered harbor, steep rocky coast, island and river mouth. Wave energy and the kind of whitewater should respond to those different settings.

Delivery sequence and remaining uncertainty

First establish a small controlled beach fixture and the signed coastal-data contract. Then implement and verify moving contact and persistent foam. Add receiver-attached caustics after that shared wave/data path is stable. Finally integrate streamed coasts and measure the target hardware. Rebuild an installer only after the actual packaged motion passes these checks; the rejected water candidate is not the completed result.

The evidence is strong on foam history, separating breaking geometry from shading, and receiver-based caustics. It is weaker on automatic coast fitting for this particular procedural world and on the cost of the proposed implementation. Those gaps require a measured prototype, not another round of cosmetic noise changes. No source reviewed establishes instant, fully realistic water for arbitrary generated terrain at a guaranteed frame rate on the target PC.

Research boundary: primary production talks, original research, maintained implementation source, Epic documentation and installed engine code were reconciled. An additional H2O in H3LL slide link returned an access error and was not used as evidence. Discovery stopped once the structural conclusions had independent primary support; the unresolved questions are now DnDRom implementation and measurement questions. Water implementation and installer acceptance remain open.